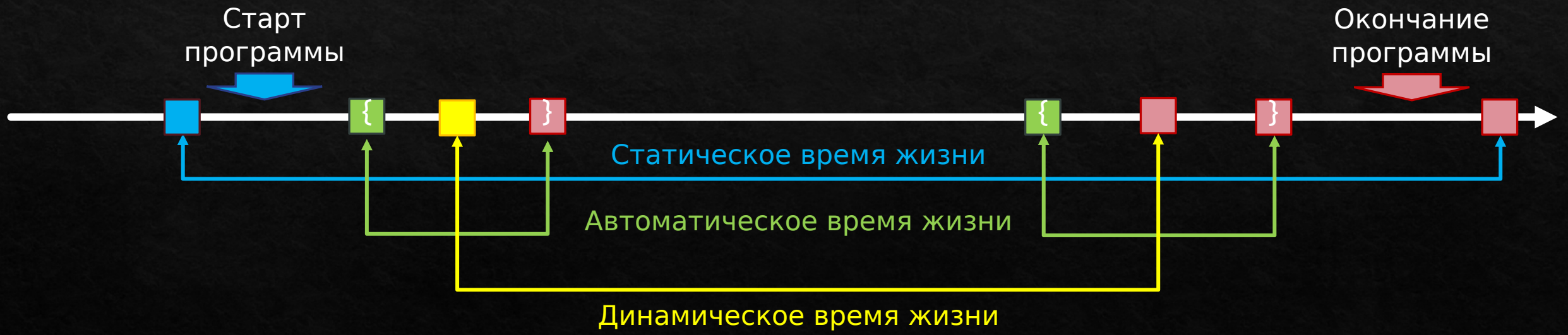
Динамическая память

создание и удаление одиночных объектов, создание и удаление массивов объектов, оператор индексирования и связь с арифметикой указателей, выделение памяти и размещение

Динамическое время жизни

В некоторых случаях мы хотим оперировать временим жизни своих объектов произвольным образом. Т.е. самостоятельно определять в какой момент в программе нужно создать объект и в какой момент этот объект больше нам не требуется. Также могут возникать ситуации, когда нам нужно, чтобы объект созданный в одном блоке кода мог быть использован вне его. Например мы хотим создать внутри функции массив и передать его в качестве результата.



Куча (heap)

Куча (heap / динамическая память) – это часть памяти, которая специально выдаётся программе (процессу) для хранения данных с неопределённым временем жизни.

В отличие от области с данными и стека, где память заполняется строго определённым образом, данные в куче динамичные и могут фрагментироваться в процессе работы.

Адреса в куче не выдаются программе (процессу) сразу, по мере необходимости необходимое количество байт можно запросить у операционной системы, которая произведёт отображение этих адресов в физические.



Создаём объекты в динамической памяти

new / delete

new expression

Если нам понадобилось выделить объект (непрерывный набор байтов) динамически, то в C++ мы можем воспользоваться специальным new expression:

```
new (std::nothrow) <typename>(<initialization>)
```

В качестве результата new expression возвращает адрес, начиная с которого в памяти находится наш объект.



new expression

Чтобы выделение памяти не выбрасывало исключений

```
new (std::nothrow) int;
```

Ключевое слово для выделения объекта в куче

Тип выделяемого в куче объекта



Выделение с инициализацией

```
new (std::nothrow) int(5);
new (std::nothrow) int{5};
```

new expression

Для того, чтобы мы могли работать с данными, которые выделены в куче, нам необходимо сохранить адрес, выданный вызовом new в некоторой именованной переменной.

```
int *i_ptr = new (std::nothrow) int(5);
```



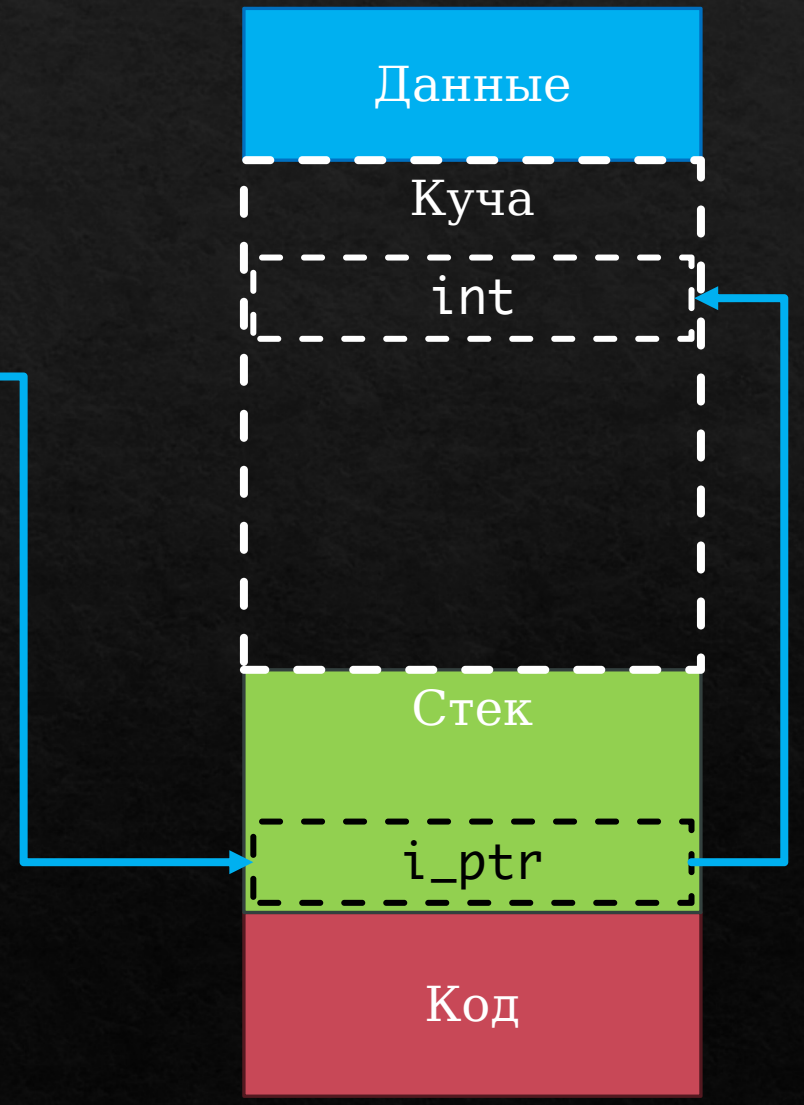
Переменная `i_ptr` сохраняет местоположение (адрес) числа типа `int`, выделенного в куче.

new expression

Посредством указателя мы можем совершать действия с объектом в динамической памяти, хотя с ним не связано никакое имя.

```
18      int *i_ptr = new (std::nothrow) int(5);
19      std::cout << *i_ptr << std::endl;
20      *i_ptr += 2;
21      std::cout << *i_ptr << std::endl;
```

```
5
7
```



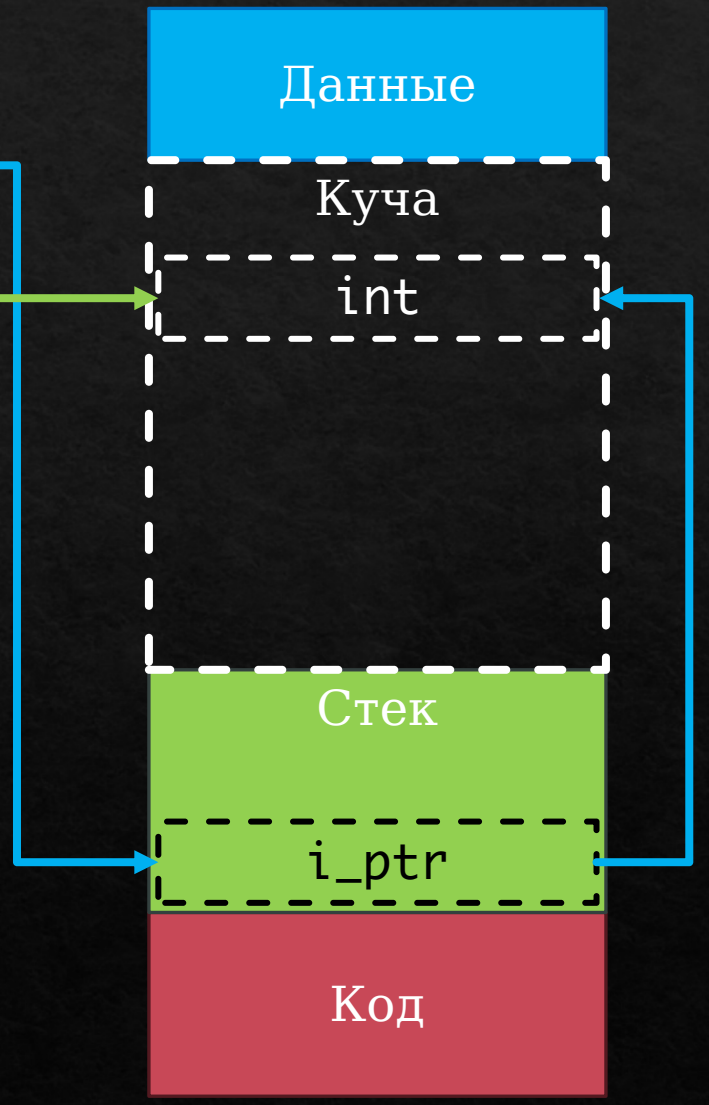
new expression

Для удобства мы можем создать ссылку на данные в куче, чтобы работать как с обычной переменной.

```
28      int *i_ptr = new (std::nothrow) int(5);
29      int &i_ref = *i_ptr;
30      std::cout << *i_ptr << " " << i_ref << std::endl;
31      std::cout << i_ptr << " " << &i_ref << std::endl;
```

```
5 5
0x600002194040 0x600002194040
```

Обратите внимание, что ссылка становится полным псевдонимом объекта в динамической памяти! Взятие адреса от ссылки даёт нам адрес элемента именно в динамической памяти.



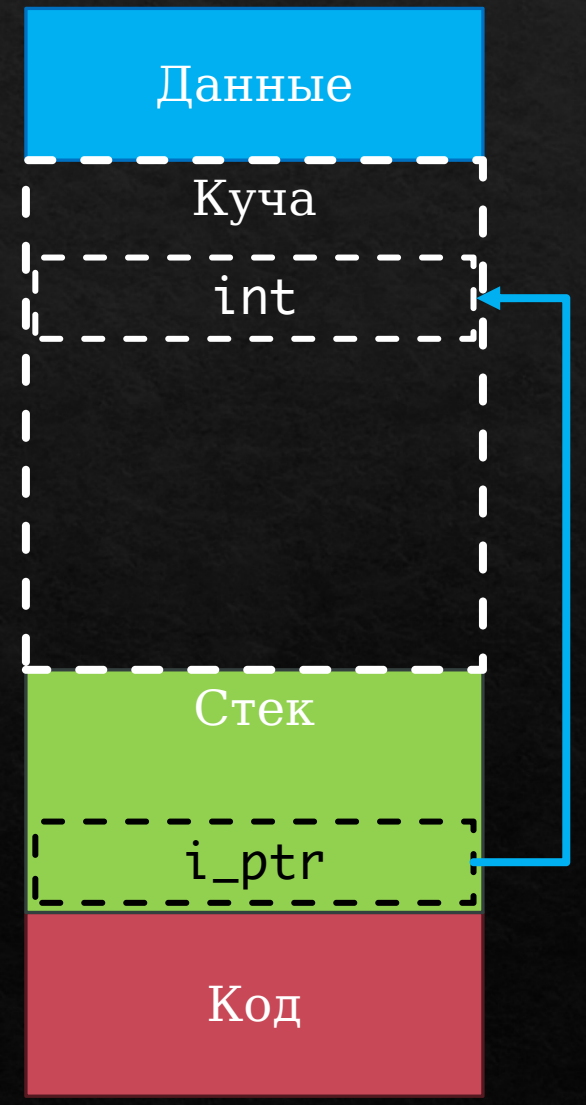
delete expression

Если объект нам больше не нужен и мы хотим освободить занятую память, то мы можем вернуть память системе. Для этого используется delete expression.

```
delete <адрес>;
```

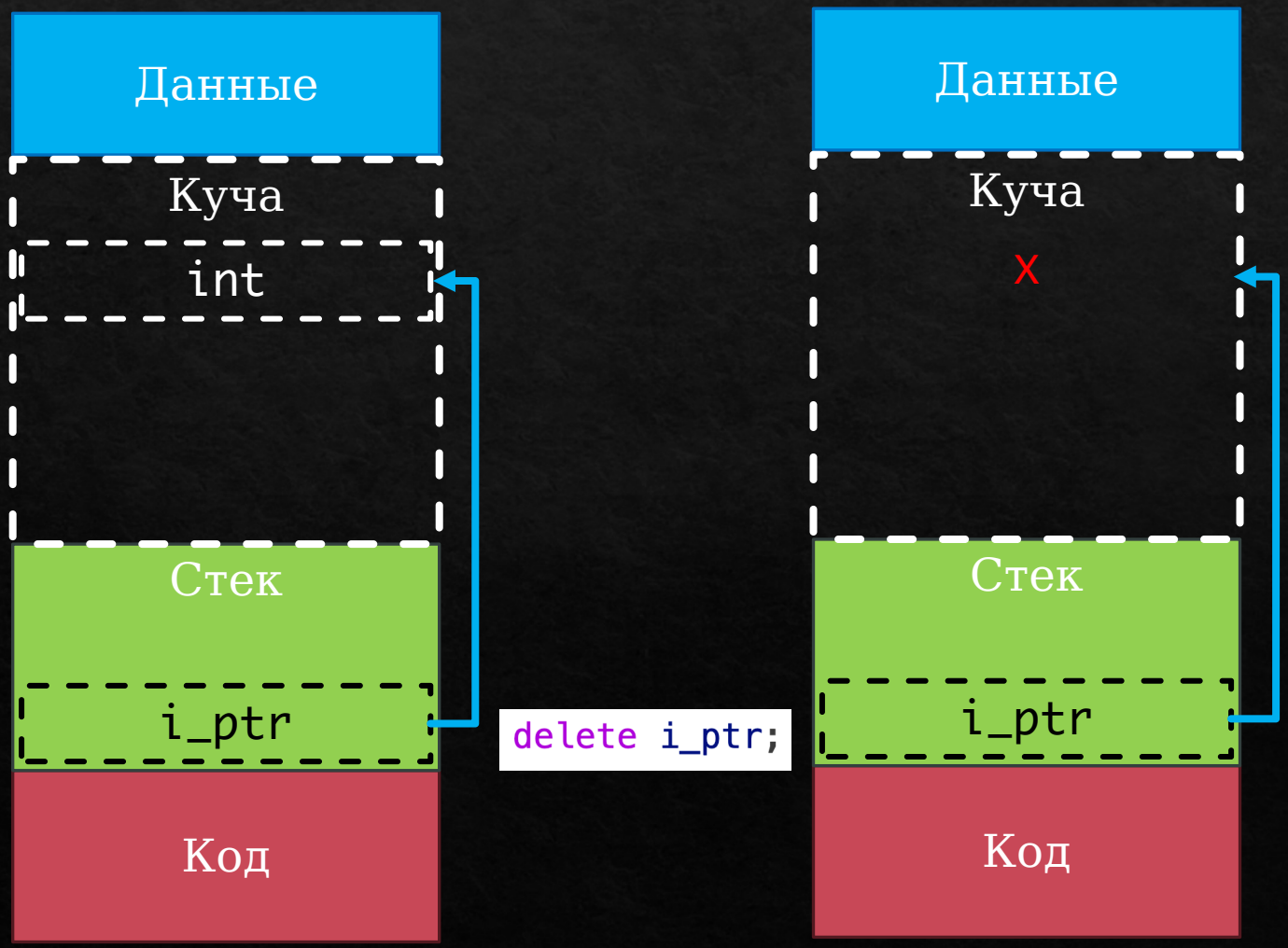
```
28      int *i_ptr = new (std::nothrow) int(5);
29
30      int &i_ref = *i_ptr;
31      std::cout << *i_ptr << " " << i_ref << std::endl;
32      std::cout << i_ptr << " " << &i_ref << std::endl;
33
34      delete i_ptr;
```

Возвращаем неиспользуемую память



delete expression

Важно понимать, что `i_ptr` (указатель на стеке, который сохраняет адрес данных в куче) – это независимая от кучи переменная и его данные никак затрагиваются вызовом delete.




```
int *i_ptr = new (std::nothrow) int(5);

std::cout << i_ptr << std::endl;

delete i_ptr;

std::cout << i_ptr << std::endl;
```

```
0x600003f64040
0x600003f64040
```

Такие указатели называются «висячими». Висячей может оказаться и ссылка, если её область видимости больше жизни объекта.

new expression failure

new expression обращается за выделением адресов к ОС. Такое действие может завершится неудачно, т.е. ОС по какой-то причине (например недостаточно памяти) не может выделить нам запрашиваемое количество адресов. В этом случае new завершается с ошибкой. Есть два варианта обработки ошибочной ситуации:

```
int *i_ptr = new int(5);
```

Если при выделении объекта используется new без отметки nothrow, то при возникновении проблем исполнение кода будет прервано исключительной ситуацией std::bad_alloc.

```
int *i_ptr = new (std::nothrow) int(5);
```

Если используется специальная нотация с гарантией отсутствия исключений, то при невозможности выделить память new expression возвращает nullptr.

new expression failure

```
int *i_ptr = new (std::nothrow) int(5);
if (nullptr == i_ptr) {
    std::cout << "Memory allocation error" << std::endl;
} else {
    std::cout << *i_ptr << std::endl;
    *i_ptr += 2;
    std::cout << *i_ptr << std::endl;
    delete i_ptr;
}
```

```
try {
    int *i_ptr = new int(5);
    std::cout << *i_ptr << std::endl;
    *i_ptr += 2;
    std::cout << *i_ptr << std::endl;
    delete i_ptr;
} catch(std::bad_alloc const &e) {
    std::cout << "Memory allocation error" << std::endl;
}
```

delete nullptr

Одно из правил «хорошего кода на C++» состоит в том, чтобы на каждый вызов new в вашей программе приходился один вызов delete. Т.е. new / delete балансируют друг друга, как скобочки.

Может возникнуть вопросы:

1. Если вызов new провален, нужно ли вызывать delete?
2. Могу ли я вызвать delete для nullptr?

Если вызов new expression провален, то нам гарантируется, что программа останется в корректном состоянии в котором она была до его вызова, вызывать delete не нужно.

При вызове delete на nullptr никакой памяти освобождено не будет и это не приведёт к ошибкам работы с памятью. Однако стандарт не определяет будет ли явным образом вызвана функция возвращения памяти (это важно для объектов, которые перегружают операторы new / delete). Но даже если функция будет вызвана, она не должна оказывать воздействие на память.

Ошибки при работе с new / delete

use uninitialized, use-after-free, wrong free, double free, no free (memory leak)

1. use uninitialized – использование памяти, которая не была выделена, например из-за игнорирования ошибки выделения памяти
2. use after free – обращение к памяти по висячему указателю или висячей ссылке, когда память уже была освобождена
3. wrong free – попытка освобождения памяти по адресу, который никогда не был выделен в куче
4. double free – повторное освобождение памяти по адресу, который уже был возвращён системе
5. no free (memory leak) – отсутствие вызова освобождения памяти и утеря адреса, по которому она была занята, что приводит к невозможности возвращения адресов системе («утечка памяти»)

Неинициализированный / непроверенный указатель

```
76  void use_uninitialized() {
77      int *uninit_ptr; // неинициализированный указатель
78      std::cout << *uninit_ptr << std::endl;
79
80      int *null_ptr = nullptr; // нулевой указатель
81      std::cout << *null_ptr << std::endl;
82
83      // непроверенный указатель
84      int *unchecked_ptr = new (std::nothrow) int(5);
85      std::cout << *unchecked_ptr << std::endl;
86  }
```

Разыменование неинициализированного или нулевого указателя приводит к неопределённому поведению (undefined behavior). Непроверенный указатель может оказаться нулевым, тогда это приведёт к неопределённому поведению.

Использование после освобождения

```
int *dangling_pointer = new (std::nothrow) int(5);
if (nullptr != dangling_pointer) {
    std::cout << *dangling_pointer << std::endl;
    delete dangling_pointer;
    *dangling_pointer += 2;
}
```

Используем освобождённую память

```
int *ptr = new (std::nothrow) int(5);
if (nullptr != ptr) {
    int &dangling_ref = *ptr;
    delete ptr;
    dangling_ref += 2;
}
```

Используем освобождённую память

Удаление по некорректному адресу

```
int i;
int *i_ptr = &i;
delete i_ptr;
```

Пытаемся вернуть системе адрес, который не принадлежит куче

```
int *ptr = new (std::nothrow) int(5);
unsigned char *uch_ptr = reinterpret_cast<unsigned char *>(ptr);
delete (uch_ptr + 1);
```

Пытаемся вернуть память по адресу, который отличается от адреса, возвращённого new

Двойное удаление

```
int *ptrA = new (std::nothrow) int(5);
int *ptrB = ptrA;
if (nullptr != ptrA) {
    // Some code goes here ...
    delete ptrA;
    delete ptrB;
}
```

Хотя ptrB другая переменная, он содержит тот же самый адрес, что и ptrA, что приводит к двойному освобождению

Утечка памяти

```
while (true) {
    int *tmp = new (std::nothrow) int(5);
}
```

На каждой итерации цикла создаётся новая переменная tmp и значение адреса безвозвратно теряется

Создаём массивы в динамической памяти

new[] / delete[]

new[] expression

Если нам понадобилось выделить несколько однотипных элемнтов, которые строго подряд расположены в памяти (например, чтобы воспользоваться этой памятью как массивом), мы используем специальный new expression:

```
new (std::nothrow) <typename>[<size>]{<initialization>}
```

В качестве результата new expression для набора элементов возвращает адрес, начиная с которого в памяти находится наш объект.

С точки зрения управления памятью весь набор элементов – это единый объект. Для всех адресов внутри выделенного набора адресов допустима адресная арифметика и сравнение указателей, включая past-the-end (первый адрес за концом выделенной памяти), хотя past-the-end не допускает разыменования.



Между объектами, которые выделены разными вызовами new, сравнение и адресная арифметика, например разность адресов, недопустимы. Это приводит к неопределённому поведению!

new[] expression примеры

```
int *one_element = new (std::nothrow) int[1] {5};

int *some_elements = new (std::nothrow) int[3] {1, 2, 3};

int *some_zero_elemetns = new (std::nothrow) int[4] {0};

int *some_uninitialized_elements = new (std::nothrow) int [5];

int *zero_elements = new (std::nothrow) int[0];
```

Один элемент

Три элемента, каждый отдельно инициализирован

Четыре элемента, все инициализированы нулём

Пять элементов не инициализированы

Выделено ноль элементов (это допустимое выделение)

При выделении нуля элементов возвращается некоторый адрес, который считается past-the-end, т.е. разыменовать этот указатель нельзя, сдвигать этот указатель нельзя, сравнивать его не с чем, но его можно и нужно передавать в функцию возвращения памяти.
Вызов new для нуля элементов также может провалиться и выбросить исключение или вернуть nullptr.

delete[] expression

Как и в случае одного элемента, при использовании new expression для набора элементов следует возвращать память системе, когда она более не требуется. Для этого нужно вызвать специальный delete[]:

```
int *one_element = new (std::nothrow) int[1];
delete[] one_element;

int *some_elements = new (std::nothrow) int[3] {1, 2, 3};
delete[] some_elements;

int *zero_elements = new (std::nothrow) int[0];
delete[] zero_elements;
```

Не путайте delete и delete[]! Первый освобождает память на один элемент, другой – на набор элементов. Вызов delete[] на памяти для одного элемента или вызов delete на памяти для многих приводят к неопределённому поведению.

Оператор индексирования

*(a + n) ⇔ a[n] ⇔ n[a]

Обращение к элементам

Если мы выделили память с помощью new expression для набора элементов, то нам гарантируется, что элементы расположены в памяти строго подряд, без пустых промежутков. Опираясь на это, мы легко можем написать обращение к элементам через адресную арифметику:

```
167     int *some_elements = new (std::nothrow) int[3] {1, 2, 3};
168     std::cout << *(some_elements + 0) << '\n'; // первый элемент
169     std::cout << *(some_elements + 1) << '\n'; // второй элемент
170     std::cout << *(some_elements + 2) << '\n'; // третий элемент
171
172     delete[] some_elements;
```

```
1
2
3
```

Напоминание: у оператора разыменования приоритет более высокий, чем у бинарного сложения, потому мы берём смещение указателя в скобки.

Обращение к элементам

Обращение к элементам, расположенным последовательно – это чрезвычайно частая операция. Так как арифметика указателей выполняется за константное время, то и обращение к элементам требует константного времени. Получается, что последовательный набор элементов можно воспринимать как массив данных.

Для удобного обращения к элементам такого «массива» в C++ (заимствовано из C) введён «синтаксический сахар»: если к указателю приписывается целое число в квадратных скобках, то это преобразуется в смещение и разыменование.

```
ptr[n] ⇔ *(ptr + n)
```

Чтобы «рассахаривание» работало, `n` должно быть целым числом.

```
int c = some_elements[float(1.0f)];
```

```
DynamicMemory/new_expression.cpp:179:26: error: array subscript is not an integer
    int c = some_elements[float(1.0f)];
                          ^~~~~~~~~~~
```

Обращение к элементам

Рассахаривание происходит и в том случае, когда к целому числу справа приписывается указатель в квадратных скобках:

```
int b = 0[some_elements];
```

Такой синтаксис унаследован из языка C, в котором квадратные скобки всегда превращаются в разыменование от суммы, сумма же не меняется от порядка слагаемых.

```
ptr[n] ⇔ *(ptr + n) ⇔ *(n + ptr) ⇔ n[ptr]
```

Смещения могут быть и отрицательными, например если мы возьмём адрес из середины массива:

```
int *some_elements = new (std::nothrow) int[3] {1, 2, 3};

int a = some_elements[0];
int b = 2[some_elements];

int *central_element_address = some_elements + 1;
std::cout << a << " " << (-1)[central_element_address] << '\n';
std::cout << b << " " << 1[central_element_address] << '\n';
```

Обращение к элементам

Приоритет	Оператор	Описание
1	::	Разрешение области видимости
2	a++ a-- *тип*() *тип*{} a() a[]	Суффиксный/постфиксный инкремент и декремент Функциональный оператор приведения типов Вызов функции Индексация
	. ->	Доступ к элементу
3	++a --a +a -a ! ~ (*тип*) *a &a sizeof co_await new new[] delete delete[]	Префиксный инкремент и декремент Унарные плюс и минус Логическое НЕ и побитовое НЕ Приведение типов в стиле C Косвенное обращение (разыменование) Взятие адреса Размер в байтах Выражение await (C++20) Динамическое распределение памяти Динамическое освобождение памяти

```
{
    int *elements = new int[2] {-2, 2};
    int *p = elements;
    std::cout << ++*p << std::endl;
    delete[] elements;
}
```

-1

```
{
    int *elements = new int[2] {-2, 2};
    int *p = elements;
    std::cout << *++p << std::endl;
    delete[] elements;
}
```

2

```
{
    int *elements = new int[2] {-2, 2};
    int *p = elements;
    std::cout << ++p[0] << " " << p[0] << std::endl;
    delete[] elements;
}
```

-1 -1

```
{
    int *elements = new int[2] {-2, 2};
    int *p = elements;
    std::cout << p++[0] << " " << p[0] << std::endl;
    delete[] elements;
}
```

-2 2

Массивы и указатели

Когда мы передаём в функцию массив с использованием синтаксиса вида <indentifier>[ ], то мы на самом деле получаем указатель на первый элемент массива. Это особое свойство массивов в языках C / C++: идентификаторы с типом массива могут неявно преобразовываться к указателю на первый элемент со связанным типом равным типу хранящихся в массиве элементов.

```
int arr[3] = {1, 2, 3};
int *ptr = arr; // не ставим взятие адреса
assert(ptr == &(arr[0]));
```

Указатель на указатель

```
int const * const * const a;
```

Указатель на указатель

В указателе можно сохранить адрес любой переменной, в том числе и другого указателя. Основная концепция достаточно очевидна: нужно определить в связанном типе указатель.

```
int i = 5; // число

int *i_ptr = &i; // указатель на число -> связанный тип int

int * * i_ptr_ptr = &i_ptr; // указатель на указатель -> связанный тип int *
```

```
std::cout << i << " " << i_ptr << " " << i_ptr_ptr << '\n';

std::cout << *i_ptr << " " << *i_ptr_ptr << '\n';

std::cout << *(*i_ptr_ptr) << '\n';
```

```
5 0x7ff7b21ad774 0x7ff7b21ad778
5 0x7ff7b21ad774
5
```

Уровень косвенности не ограничен. Мы можем сделать указатель, который сохраняет адрес указателя на указатель. И т.д.

```
int * * * i_ptr_ptr_ptr = &i_ptr_ptr;
```

Зачем это нам нужно?

Используя такой подход можно организовать похожие по синтаксису на многомерные массивы структуры данных.

Для начала создадим массив адресов в динамической памяти:

```
int* *arr_of_arrs = new (std::nothrow) int*[3] {nullptr, nullptr, nullptr};
```

Теперь для каждого элемента массива сохраним в нём адрес отдельного массива элементов типа `int`:

```
for (std::size_t idx = 0u; idx != 3u; ++idx)
    arr_of_arrs[idx] = new (std::nothrow) int[4];
```

Обратиться к некоторому элементу типа `int` через указатель `arr_of_arrs` можно вызвав оператор индексирования два раза:

```
int* sub_array = arr_of_arrs[0];
sub_array[0] = 1;
```

```
// эквивалентно
arr_of_arrs[0][0] = 1;
```

```
// эквивалентно
arr_of_arrs[0][0] = 1;
```

Константность при работе с множественной косвенностью

При работе с множественной константностью могут возникнуть проблемы с неочевидными преобразованиями.

```
int i = 5;
int *ptr = &i;
int **ptr_ptr = &ptr;

int const *cv_ptr = ptr; // good

int const * * cv_ptr_ptr = ptr_ptr; // bad
```

```
int * const ptr_cv = ptr; // good
int * * const ptr_ptr_cv = ptr_ptr; // good
```

```
DynamicMemory/ptr_to_ptr.cpp:74:19: error: cannot initialize a variable of type 'const int **' with an lvalue of type
 'int **'
    int const * * cv_ptr_ptr = ptr_ptr;
                  ^            ~~~~~~~
```

```
const int n = 42;
const int* cp = &n;

int* p;
int** a = &p;

const int** b;
b = a;  // если бы это компилировалось
*b = cp;  // то это присваивание, верное для b и для cp
*p = 84;  // привело бы к возможности изменить константу через p
```

```
int const * const * cv_ptr_cv_ptr = ptr_ptr; // good
```

Для такого указателя запрещено косвенное изменение даже для одного разыменования.